EMPIRE BUILDER

FROM ZERO TO TYCOON

FULL GAME DEVELOPMENT GUIDE

Game Design • Financial Literacy • Unity Production • Systems • UI/UX • Testing • Submission

Core fantasy

Start with limited cash. Buy income-producing assets. Manage repairs and risk. Reinvest profits. Build a thriving business empire.

Prepared as a practical blueprint for a solo developer or small team

Version 1.0 | October 2026

How to Use This Guide

This document turns your business-investment game idea into a production plan. It covers the playable concept, financial systems, a realistic first release, Unity project architecture, data models, implementation order, testing, and a game-jam submission package. Use it as a living design document: build the MVP first, test it, then expand.

Recommended default decisions

- **Engine:** Unity 6, C#.
- **Camera:** isometric / elevated 3D camera for clear city management.
- **Gameplay:** hybrid management: click assets and use focused interaction panels; optional close-up inspection later.
- **Setting:** a compact modern city with a few distinct districts. Art can be stylized rather than photorealistic.
- **Platform:** Windows PC for the first prototype; consider WebGL only after performance and memory testing.
- **First release:** one district, three asset types, one financial dashboard, daily simulation, repairs, random events, one clear win goal.
- **Design principle:** every major mechanic should support a meaningful financial decision.

Contents

1. High Concept and Game Pillars
2. Target Player, Learning Goals and Core Loop
3. MVP Scope and Feature Priorities
4. Assets, Businesses and Economy
5. Financial Simulation Rules
6. Progression, Events and Challenges
7. UI/UX and Player Feedback
8. Unity Technical Architecture
9. Suggested Project and Folder Structure
10. Core Data Models and Example Logic
11. Step-by-Step Development Roadmap
12. Art, Audio and Content Pipeline
13. Testing and Balancing
14. Game-Jam Submission Package
15. Risks, Scope Control and Future Expansion

Appendix A. Example Starting Economy

Appendix B. Sample Two-Minute Gameplay Video

Appendix C. Final Checklists and Glossary

1. High Concept and Game Pillars

Working title: Empire Builder: From Zero to Tycoon. The title is provisional; check availability and distinctiveness before publishing.

One-sentence pitch: Start with limited cash, buy vehicles and property, earn income, maintain assets, handle financial surprises, and reinvest carefully to build a profitable business empire.

Genre and player promise

A 3D tycoon / investment management simulation where the player grows a portfolio of vehicles, land, rental buildings and eventually resorts. The game should make financial cause and effect visible: buying something changes cash, assets can produce income, operations create costs, neglect creates repair bills, and risky decisions can create either growth or losses.

Design pillars

- **Readable finance:** show purchase price, income, expenses, net profit, payback estimate, condition and risk before the player commits.
- **Visible growth:** assets and the city change visibly as the business expands.
- **Meaningful trade-offs:** the player cannot buy everything immediately; liquidity, maintenance and risk matter.
- **Short feedback loops:** each in-game day provides a clear report and at least one useful decision.
- **Learning without lectures:** the game teaches through outcomes, comparisons, tooltips and optional short explanations.

Player fantasy

The player begins as a small investor. They may buy a taxi, purchase an inexpensive plot, or acquire a modest rental building. They learn to distinguish revenue from profit, keep enough cash for repairs, and reinvest only when the business can support it. Later they operate multiple assets and unlock more complex businesses.

What the game is not in the MVP

- Not a full open-world driving simulator.
- Not a multiplayer economy or online trading platform.
- Not a complex real-world accounting package.
- Not a city-builder with hundreds of simulated citizens.
- Not a promise that every investment always makes money.

These limits protect the central experience and make a polished jam build achievable.

2. Target Player, Learning Goals and Core Loop

Target audience

- Players who enjoy tycoon, idle-management, business and incremental games.
- Teen and adult players who are curious about business and money management.
- Game-jam judges looking for a clear connection between gameplay and financial literacy.

Learning outcomes

Learning goal	How gameplay teaches it	Evidence in the game
Revenue vs. profit	Income is reduced by wages, fuel, repairs and other costs.	Daily report shows gross revenue, costs and net profit.
ROI / payback	Compare assets by price, net income and expected recovery time.	Asset card displays estimated payback and assumptions.
Cash-flow planning	A profitable business can still struggle if cash is spent too early.	Cash-on-hand and upcoming obligations are visible.
Emergency savings	Unexpected repairs can disrupt unprepared businesses.	Reserve protects the player from selling assets in a crisis.
Debt and interest	Loans accelerate expansion but require repayment.	Loan panel shows principal, interest, installment and risk.
Diversification	Different assets react differently to market events.	Portfolio summary groups assets and exposure by type.
Maintenance	Preventive repair costs money now but can reduce larger losses later.	Condition affects revenue, breakdown probability and repair cost.

The core loop

1. **Review:** read the daily financial report and asset alerts.
2. **Choose:** inspect available assets and compare expected returns, expenses and risk.
3. **Invest:** purchase an asset, improve a property, or keep cash in reserve.
4. **Operate:** assets generate revenue and incur operating costs.
5. **Maintain:** repair or service assets, hire help if available, and manage downtime.
6. **Resolve:** react to a market change, customer issue or random event.
7. **Evaluate:** review profit, cash, debt and net worth; decide what to do next.

A good session should repeatedly ask the player: “Can I afford this, what could it earn, what could go wrong, and will I still have enough cash afterward?”

3. MVP Scope and Feature Priorities

Build a complete small experience before creating many asset types. The MVP should have a beginning, understandable decisions, visible outcomes and a win condition.

Priority	Feature	Acceptance criteria
P0 — required	Cash and net worth	Cash, asset value, liabilities and net worth update correctly.
P0 — required	Asset shop	Player can inspect price, income, costs, condition and buy only if affordable.
P0 — required	Three asset types	Taxi, rental building and vacant land have different economics.
P0 — required	Daily simulation	One day calculates revenue, costs, repairs and reports once.
P0 — required	Maintenance	Condition declines; repair costs cash and can restore condition.
P0 — required	Financial dashboard	Shows cash, daily net profit, assets, liabilities and net worth.
P0 — required	Save/load	Save game state locally and resume without duplicating a day.
P0 — required	Learning feedback	At least three contextual explanations for key financial concepts.
P1 — next	Random events	At least three balanced events with clear consequences.
P1 — next	Asset upgrades	Upgrade changes income, cost or risk in a readable way.
P1 — next	Loans	Borrowing, interest and repayments are understandable and testable.
P2 — later	Resorts and staff	Add only after the first economy is stable.
P2 — later	Walkable world / driving	Optional immersive layer; not necessary for the financial core.

Do not start with all features

Avoid simultaneous development of driving, construction animations, multiple districts, staff AI, resort occupancy, loans, weather, complex market simulation and a huge inventory. First prove that purchasing an asset, receiving a daily report and choosing the next investment is satisfying.

Definition of MVP complete

- A new player can understand the objective within two minutes.
- The player can buy, operate, maintain and sell or keep at least three asset types.
- A day can be advanced without duplicate or missing financial transactions.
- The player can recover from a bad choice through sensible play, rather than becoming permanently stuck.
- The game saves and loads reliably.
- The build has no known critical bugs and includes clear controls.

4. Assets, Businesses and Economy

Asset categories for later expansion

Asset	Income source	Recurring costs	Risk / gameplay decision
Taxi / car	Fares per day	Fuel, driver, service, insurance	Downtime, wear, variable demand
Delivery van / truck	Delivery contracts	Fuel, wages, maintenance	Contract deadlines and repair risk
Vacant land	Usually none until developed; potential resale value	Taxes / holding costs if enabled	Market appreciation is uncertain
Rental building	Rent from occupied units	Repairs, utilities, management, tax	Vacancy and major repairs
Shop / café	Sales margin	Inventory, wages, utilities	Demand changes and stock waste
Hotel / resort	Bookings and services	Staff, utilities, cleaning, repairs	Seasonality and occupancy
Warehouse	Rent or logistics fees	Maintenance, staff, security	Demand and operating capacity

MVP asset definitions

Asset	Illustrative purchase price	Gross income/day	Operating cost/day	Expected net/day before major repairs
Taxi	$5,000	$120	$45	$75
Small rental building	$18,000	$260	$110	$150
Vacant plot	$8,000	$0	$5	-$5 holding cost

These are placeholder game-balance values only. They are not real market estimates. Test them with play sessions and adjust them until every option has a reason to exist.

Asset information panel

- Purchase price and whether the player can afford it.
- Expected gross income per day and expected operating cost.
- Estimated net income and payback time, clearly marked as an estimate.
- Current condition and likely repair cost.
- Demand / risk indicator and any assumptions behind the estimate.
- Action buttons: Buy, Repair, Upgrade, Sell, Close/Resume operations where relevant.

Asset state machine

Each asset should have a simple state such as AvailableForPurchase, Operating, NeedsRepair, UnderRepair, Inactive or Sold. Define which actions are legal in each state. For example, an asset under repair should not earn full income unless the design explicitly allows partial operation.

5. Financial Simulation Rules

Core values

Value	Meaning
Cash	Money immediately available to spend.
Gross revenue	Money earned before costs.
Operating expenses	Fuel, wages, utilities, routine service and other recurring costs.
Net operating income	Gross revenue minus operating expenses; define treatment of taxes and debt separately.
Repair expense	One-off or scheduled cost to restore an asset.
Liabilities	Outstanding loan principal and other unpaid obligations.
Asset market value	Current estimated sale value of owned assets.
Net worth	Cash + current asset values - liabilities.
Liquidity	Cash available to cover upcoming expenses and shocks.

Recommended calculation order for each in-game day

1. Freeze the current set of operating assets for the day's calculation.
2. Determine each asset's gross revenue using its type, demand, condition and upgrades.
3. Calculate operating expenses for each asset.
4. Apply scheduled maintenance and breakdown events according to the chosen rules.
5. Apply taxes or debt installments if included; show them as separate report lines.
6. Update cash exactly once using the sum of all transactions.
7. Update condition, market values and any progression milestones.
8. Create a daily report from the completed transactions, then save.

Key formulas

Net operating income = Gross revenue – Operating expenses
Daily cash change = Cash received – Cash paid
Net worth = Cash + Asset market values – Outstanding liabilities
Simple payback estimate = Purchase price ÷ Expected daily net income, only when net income is positive.

Important accounting distinction

Profit and cash change are not always the same. Buying an asset reduces cash but adds an asset to the portfolio. Borrowing increases cash and liabilities together. Selling an asset converts it to cash; it should not create free wealth. For the MVP, keep accounting simple but make every transaction explicit.

Prevent common economy exploits

- Never let the same day's income run twice because the player clicked twice or loaded a save.
- Do not allow purchases when cash is insufficient unless a loan feature is deliberately used.
- Prevent selling an asset that is already sold or under an incompatible operation.
- Avoid infinite profit loops caused by buying and immediately reselling at a higher price without a market rule.
- Use a consistent policy for asset value, depreciation, repairs and sale fees.

- Use a single simulation service as the authority for money changes; UI buttons should request transactions rather than directly editing cash.

6. Progression, Events and Challenges

Progression model

Stage	Illustrative net-worth milestone	Unlock
Beginner	$0–$25,000	Taxi, vacant land, basic dashboard
Entrepreneur	$25,000–$100,000	Second vehicle, rental building, repair upgrades
Business owner	$100,000–$500,000	More properties, staff, basic loans
Investor	$500,000–$2 million	Larger buildings, warehouses, expanded market
Tycoon	$2 million+	Hotels/resorts, multiple districts, advanced challenges

Use net worth plus financial health, not only total cash earned. A player who earns a lot but has excessive debt should not automatically count as financially successful.

Random event examples

Event	Choice	Consequence
Vehicle warning light	Pay for preventive service or keep operating	Service costs cash now; ignoring it raises breakdown risk.
Slow tourist season	Cut expenses, promote offers or maintain normal operations	Different short-term costs and occupancy outcomes.
Unexpected building leak	Repair immediately or delay	Immediate cost versus possible additional damage / lost rent.
Land development offer	Sell, hold or develop	Trade liquidity for uncertain future value.
New delivery contract	Accept or decline	Potential revenue with fuel, time and vehicle-condition trade-offs.
Suspicious investment pitch	Investigate or invest immediately	Teach warning signs, promised returns and due diligence.

Make events fair

- Give the player enough information to make a reasoned decision.
- Avoid random losses that instantly destroy a well-managed run.
- Let preparation matter: good condition, reserves and diversification should improve resilience.
- Explain the lesson after the outcome without shaming the player.
- Use seeded random values during testing so bugs can be reproduced.

Challenge examples

- Reach $50,000 net worth without taking a loan.
- Maintain a three-day emergency reserve for 10 consecutive days.
- Expand to three asset types while keeping positive net cash flow.
- Survive a low-demand week without selling your core asset.
- Compare two investment offers and select the better risk-adjusted option.

7. UI/UX and Player Feedback

Main HUD

UI region	Display
Top bar	Cash, today's net income, net worth, day number, pause / settings
Left panel	District / asset categories / available opportunities
Center	City view with selectable assets, lots and construction sites
Right panel	Selected asset details, condition, expected return and actions
Bottom / modal	Advance Day, event decisions, daily report and tutorial prompts

Essential screens

- **Start screen:** New Game, Continue, Settings, Credits.
- **City screen:** map, assets, purchase points and day controls.
- **Asset detail:** financial metrics, condition, maintenance, upgrade and sale options.
- **Investment shop:** comparable opportunities with costs and risk.
- **Daily report:** revenue, expenses, repair costs, debt payments, net cash change and lesson.
- **Portfolio:** assets by type, market value, income contribution and concentration risk.
- **Settings / pause:** audio, graphics, controls, save/quit.

Financial readability rules

- Never show income without also making costs visible.
- Label all values with currency and time period, e.g. "$75/day".
- Use green/red only as supporting signals; also use labels and icons for accessibility.
- Distinguish estimate from guaranteed value.
- Confirm irreversible actions such as selling a valuable asset.
- Explain unfamiliar terms in short optional tooltips.

Sample daily report

Line item	Example
Taxi fares	+$120
Rental income	+$260
Fuel and driver	-$45
Building operating costs	-$110
Unexpected repair	-$80
Net cash change	+$145
Lesson	A business can make money overall while one repair still reduces today's cash flow.

8. Unity Technical Architecture

Recommended approach: data-driven, modular C# systems. Avoid placing all logic in one MonoBehaviour. Keep the economy independent from visual assets so the same rules work whether the player clicks a 3D car, a map icon or a list item.

Main systems

System / class	Responsibility	Should not do
GameManager	Own game state and coordinate new game, day advancement and save/load.	Contain every business rule.
EconomyService	Validate and apply money transactions; calculate cash and net worth.	Control UI visuals.
SimulationService	Advance exactly one in-game day and process assets/events in order.	Read arbitrary UI button state.
AssetDefinition (ScriptableObject)	Static data: type, base price, income, costs, condition settings.	Store player-specific changing state.
OwnedAsset	Runtime state for one purchased asset.	Act as the global source of every UI value.
AssetManager	Create, query, sell and update owned assets.	Directly manipulate unrelated UI.
MaintenanceService	Calculate condition, repair price and downtime.	Run duplicate daily ticks.
EventService	Choose and resolve events; support deterministic testing.	Change cash without a recorded transaction.
SaveService	Serialize and validate persistent state.	Serialize scene GameObjects as the entire save format.
UI presenters	Display data and send player commands to services.	Implement economy calculations in button handlers.

Data vs. scene objects

Use ScriptableObjects for fixed asset definitions, such as a taxi's base price. Use serializable runtime classes for owned asset instances, such as Taxi #01 with 72% condition. The 3D GameObject should display or represent the runtime asset, not be the only place its financial state exists.

Time model

- Use an explicit in-game day counter, not the computer's real date, for the simulation.
- Advance the economy only when the player chooses Advance Day or an equivalent action.
- If you later add real-time income, separate it from the turn-based simulation and test save/load carefully.
- Use a transaction ID or day index so a save/load cycle cannot apply the same daily income twice.

Unity implementation advice

- Use prefabs for selectable asset visuals and UI panels.
- Use interfaces or events to notify the UI when game state changes.
- Keep all financial arithmetic in one tested service.
- Use object pooling only if profiling shows a need; don't optimize prematurely.
- Use Unity's Input System if you need keyboard, mouse and controller support.
- Commit to version control frequently and keep a known-working build.

9. Suggested Project and Folder Structure

A practical structure for a Unity project:

```
Assets/
_Game/
Art/
Materials/
Models/
Textures/
Audio/
Data/
AssetDefinitions/
Events/
Upgrades/
Prefabs/
Assets/
Environment/
UI/
Scenes/
Boot.unity
MainMenu.unity
City.unity
Scripts/
Core/
Economy/
Assets/
Simulation/
Maintenance/
Events/
SaveLoad/
UI/
World/
Settings/
Tests/
ThirdParty/
```

Suggested scene responsibilities

- **Boot:** initialize configuration and services; route to menu.
- **MainMenu:** start, continue, settings and credits.
- **City:** world, selectable asset representations, HUD and interaction panels.

Naming conventions

- C# classes and public methods: PascalCase, e.g. SimulationService.AdvanceDay().
- Private fields: choose one consistent style, e.g. _currentDay.
- ScriptableObject assets: Asset_Taxi_Basic, Asset_RentalBuilding_Small.
- Events: Event_VehicleBreakdown, Event_ResortLowSeason.
- Keep scene and prefab names descriptive; avoid names like New GameObject (17).

Version control

- Use Git with Unity's recommended .gitignore and visible-text serialization settings.
- Commit a working state before large refactors or importing major assets.
- Do not commit secrets, API keys or personal account credentials.
- Record third-party asset licenses and retain attribution requirements.

10. Core Data Models and Example Logic

The following snippets are starter patterns, not a complete drop-in game. Adapt names and references to your project, add validation, and test each system before connecting it to the UI.

A. AssetDefinition ScriptableObject

```
using UnityEngine;

public enum AssetType { Vehicle, Land, Building, Resort }

[CreateAssetMenu(menuName = "Empire Builder/Asset Definition")]
public class AssetDefinition : ScriptableObject
{
public string assetId;
public string displayName;
public AssetType assetType;
[Min(0)] public double purchasePrice;
[Min(0)] public double grossIncomePerDay;
[Min(0)] public double operatingCostPerDay;
[Range(0, 100)] public float startingCondition = 100f;
[Min(0)] public double baseRepairCost = 100;
}
```

Create one asset definition per item through Assets → Create → Empire Builder → Asset Definition. Fill the fields in the Inspector. Static definitions should not be edited at runtime to represent damage to a specific purchased asset.

B. Runtime owned asset

```
using System;

[Serializable]
public class OwnedAsset
{
public string instanceId;
public string definitionId;
public float condition = 100f;
public bool isOperating = true;
public int purchaseDay;
public double currentMarketValue;
}
```

C. EconomyService principles

- Create one method such as TryPurchaseAsset(definition) that checks affordability, subtracts the price and records a transaction.
- Create one method such as ApplyTransaction(amount, reason, transactionId) that is the only supported path for changing cash.
- Use positive amounts for inflows and negative amounts for outflows, or use explicit debit/credit fields; do not mix conventions.
- Reject NaN, infinity, negative prices and duplicate transaction IDs.
- Keep an immutable or append-only transaction log during a run so daily reports can be generated and bugs investigated.

D. Example daily simulation pseudocode

```
if (dayAlreadyProcessed) return;
var transactions = new List<Transaction>();
foreach (var asset in ownedAssets)
{
if (!asset.isOperating || asset.condition <= 0) continue;
transactions.Add(CreateRevenue(asset));
transactions.Add(CreateOperatingCost(asset));
UpdateCondition(asset);
}
ResolveScheduledMaintenanceAndEvents(transactions);
economy.ApplyBatch(transactions, dayId);
currentDay++;
MarkDayProcessed(dayId);
CreateDailyReport();
SaveGame();
```

Do not copy the pseudocode as-is: the real implementation must define transaction types, day IDs, error handling, event order, condition logic, and atomic save behavior.

E. Maintenance formula example

```
repairPrice = baseRepairCost × (1 + (100 − condition) / 100)
newCondition = 100 after a successful full repair
```

This formula makes repairs more expensive as condition drops. Tune it through playtesting; you may want partial repairs or different rules per asset type.

F. Example net worth calculation

```
netWorth = cash + sum(ownedAssets.currentMarketValue) − totalOutstandingDebt
```

Ensure every owned asset uses a consistent market-value policy. Purchase price, market value and resale price are not automatically the same.

11. Step-by-Step Development Roadmap

Use milestone gates. Move on only when the current milestone works in a playable build. A solo developer's schedule varies significantly with experience, art expectations and available time; the weeks below are planning estimates, not a guarantee.

Milestone	Tasks	Exit condition
0. One-page plan	Lock target platform, camera, core loop, learning goals and MVP.	Scope fits a short playable prototype.
1. Greybox city	Create a ground plane, roads, three plots and placeholder assets.	Player can select objects and see a detail panel.
2. Economy core	Cash, transactions, purchase validation, daily income and expenses.	Automated tests verify calculations.
3. Daily loop	Advance Day, report screen, condition decay and repairs.	One full day works without duplicate payments.
4. Progression	Net-worth milestones, unlocks and win condition.	Player can reach a clear ending.
5. Learning layer	Tooltips, comparisons, short event explanations.	Each learning goal appears in gameplay.
6. Save/load	Persist day, cash, assets, condition, transactions and unlocks.	Save/load round trip passes tests.
7. Presentation	Replace placeholders, polish UI, add audio and feedback.	Game is readable and visually coherent.
8. QA and build	Fix critical bugs, test balance, export clean build.	Fresh install works on a second test machine if available.
9. Submission pack	Game page, gameplay video, GDD, budget and disclosures.	All links and files are verified before submission.

Recommended daily workflow

1. Write one small, testable task.
2. Implement the simplest version.
3. Play the build immediately.
4. Test edge cases such as insufficient cash, zero income, broken asset and repeated clicks.
5. Commit the working version to Git.
6. Record what changed and the next task.

A playable prototype is more valuable than a large unfinished feature list. If time gets short, cut asset variety and visual complexity before cutting the daily economy loop or financial learning.

12. Art, Audio and Content Pipeline

Visual direction

- Choose stylized low-poly or clean semi-realistic 3D to keep assets readable and production manageable.
- Use a consistent scale for cars, buildings, roads and clickable plots.
- Use material and color variations to communicate asset tier or condition.
- Show construction progress with a small number of visible stages rather than complex construction simulation.
- Make the city attractive at the management camera distance; small details that cannot be seen are lower priority.

Asset list for the MVP

Asset group	Minimum content
Environment	Road segment, sidewalk, ground, trees, lights, simple sky / lighting.
Vehicle	One taxi model, optional damaged / repaired visual state.
Building	One rental building with a clear selection highlight.
Land	Three purchasable plots with vacant / owned state.
UI	Icons for money, income, costs, condition, repair, land, vehicle and reports.
Effects	Purchase confirmation, cash change, repair effect, warning indicator.
Audio	UI click, purchase, repair, day report, warning and ambient city loop.

Using external or AI-generated content

- Check each asset's license and whether commercial use is allowed.
- Keep a source / license spreadsheet for third-party models, fonts, sounds and textures.
- Avoid logos or branded vehicle designs unless you have permission.
- Follow the jam's current rules for AI disclosure and any restrictions on generated content.
- Do not use an asset pack if it makes the project too large or introduces dependencies you cannot debug.

Polish that has high value

- Clear purchase and repair animations.
- A satisfying daily report with understandable numbers.
- A visible change when condition drops or a repair completes.
- Small camera movement and selection feedback.
- A before/after city view showing progress.

13. Testing and Balancing

Test categories

Test	Example cases
Economy math	Purchase, sale, income, expenses, repair, debt and net-worth calculations.
State transitions	Operating → NeedsRepair → UnderRepair → Operating; Sold cannot earn income.
Boundary cases	Cash exactly equals price, cash one unit below price, zero income, zero condition.
Day progression	Repeated click, pause, save/load before and after advancing day.
Events	Every choice applies its intended costs and outcomes once.
Save/load	Missing or old fields, corrupted file, new game, continue game.
UI	Text fits, numbers are readable, disabled buttons explain why.
Performance	Stable frame rate with the expected number of assets and UI elements.
Accessibility	Icons have labels; meaning is not conveyed only by color; text size is legible.

Balance targets

- The first meaningful purchase should be reachable quickly.
- At least two starting strategies should be viable, not just one obvious optimal asset.
- The player should usually be able to understand why a day was profitable or unprofitable.
- Maintenance should matter without becoming repetitive busywork.
- A bad decision should create a challenge, not necessarily end the run.
- The end goal should be reachable in a short demo without grinding for an excessive amount of time.

Telemetry for local testing

During development, log day number, cash, net worth, asset count, revenue, expenses, repair events and event choices. Keep this local and avoid collecting personal information. Use the logs to find exploits and overly dominant strategies.

Release checklist

- No critical errors or missing references in the Console.
- The first-time user can start and complete a run.
- Build opens on the target platform without the Unity Editor.
- All buttons work and the controls are explained.
- Save/load and daily advancement have been tested repeatedly.
- Credits and licenses are included where required.

14. Game-Jam Submission Package

For the Zoud GameLab submission, verify the latest official jam page and FAQ for exact deadlines, form fields, funding eligibility and file requirements. The following is a practical preparation checklist, not a substitute for the organizers' current rules.

A. Game page

- Title and one-sentence pitch.
- Short description explaining the financial decisions players make.
- Screenshots showing the city, asset panel and daily report.
- Playable build or browser version if supported and tested.
- Controls and accessibility notes.
- Credits and asset licenses.
- AI-tool disclosure if required by the jam.

Suggested short description

Begin with limited capital and build a business empire one decision at a time. Buy vehicles, land and rental properties, earn daily income, manage operating costs, repair assets and prepare for unexpected setbacks. Compare investments, protect your cash reserves and grow sustainably in this financial-literacy tycoon simulation.

B. Game Design Document (GDD)

- High concept, target audience and learning goals.
- Core loop and controls.
- Asset categories, economy formulas and event rules.
- UI sketches or screenshots.
- MVP scope and future roadmap.
- Risks, assumptions and development plan.

C. Two-minute gameplay video

Show real gameplay from the submitted build. Use readable capture resolution, hide development clutter, ensure text is legible and narrate only what the footage demonstrates. A suggested timed structure appears in Appendix B.

D. Game budget

Prepare a transparent budget that distinguishes money already spent, cash expenses still required, and estimated labor value. List asset purchases, software or services, audio, testing, marketing and any contractor work. If you value your own time, state the assumed hours and hourly rate; do not present estimates as invoices or guaranteed costs.

E. Before clicking submit

1. Read the current jam rules and FAQ directly.
2. Confirm your itch.io project is entered in the correct jam.
3. Verify the final build and all supporting links in a private/incognito window if possible.
4. Check the video opens and is exactly the version you intend to submit.
5. Check GDD and budget files open correctly.

6 Keep a backup copy of the build and submission package.

15. Risks, Scope Control and Future Expansion

Main production risks

Risk	Why it hurts	Mitigation
Feature creep	Too many asset types delay the playable core.	Lock MVP and keep all extras in a later list.
Unclear finance	Players cannot understand why money changes.	Use transaction logs and a daily report.
One dominant strategy	Players buy only one asset type.	Tune prices, risk, costs and unlocks with playtests.
Overly punitive random events	Players feel outcomes are unfair.	Telegraph risks and reward preparation.
Save duplication bug	Players can repeatedly collect income.	Use day IDs and idempotent simulation processing.
Visual workload	City art takes more time than mechanics.	Use stylized assets and a small district.
Unclear learning value	Game feels like generic idle income.	Tie choices to explicit financial concepts and challenges.

Future expansions after a stable MVP

- Resorts with occupancy, seasonality, staff and customer satisfaction.
- Trucks and logistics contracts.
- Construction projects with budgets, schedules and cost overruns.
- Loans, interest rates, credit risk and debt restructuring.
- More city districts with different land prices and demand.
- Hiring managers and mechanics to automate routine work.
- Scenario mode with learning objectives and performance summaries.
- Optional sandbox mode with adjustable starting cash and difficulty.

North-star rule

Every new feature should create a new decision, deepen a financial lesson, or make the existing growth loop more satisfying. If it does none of these, it probably does not belong in the next milestone.

Appendix A. Example Starting Economy

This example is designed to illustrate the trade-off between earning assets and keeping cash available. It is not a real-world financial forecast.

Starting asset / action	Cash impact	Expected daily net	Strategic trade-off
Start with cash	$10,000	$0	Maximum flexibility; no income yet.
Buy taxi	-$5,000	+$75	Income starts, but vehicle wear and breakdown risk exist.
Buy vacant plot	-$8,000	-$5	Potential future development or appreciation, but ties up cash.
Buy both	Not affordable at start	Varies	Player must choose or progress first.

Example first decision

With $10,000, the player could buy the $5,000 taxi and keep $5,000 in reserve. If the taxi produces $75 net per day before major repairs, the player earns income while retaining liquidity. Buying land instead may offer future value but does not immediately generate income. Neither choice should always be best; the game should teach the player to compare time horizon, costs and risk.

Illustrative balance rules

- Keep enough starting cash for at least one plausible unexpected cost.
- Make land useful through future development or sale, not through guaranteed appreciation.
- Use clear estimates rather than falsely precise promises.
- Balance by playtesting multiple strategies, not only calculating in a spreadsheet.

Appendix B. Sample Two-Minute Gameplay Video

Time	Footage	Narration / on-screen point
0:00–0:10	Show title, city and starting cash.	“Start with limited capital and build a sustainable business empire.”
0:10–0:30	Open investment shop and compare taxi vs. land.	Show purchase price, expected net income, costs and risk.
0:30–0:45	Buy the taxi and show it in the city.	“The asset now produces income but needs maintenance.”
0:45–1:05	Advance a day; show revenue and operating costs.	Explain revenue versus profit through the report.
1:05–1:25	Show a repair warning and pay for maintenance.	Show why preventive maintenance protects future income.
1:25–1:45	Handle an event or compare a second investment.	Demonstrate risk, emergency cash or diversification.
1:45–2:00	Show net worth, portfolio and next goal.	Close with the learning goal and current build status.

Capture checklist

- Record a real, stable build—not editor windows unless essential.
- Use 1080p if available and keep HUD text readable.
- Make sure the mouse cursor does not obscure key values.
- Avoid long loading screens, repeated menus and silent dead time.
- Show at least one meaningful financial decision and its consequence.
- Keep a backup video and verify the final link.

Appendix C. Final Checklists and Glossary

Developer checklist

- [ ] One-sentence pitch and MVP scope locked.
- [ ] Core loop playable from start to finish.
- [ ] Three asset types implemented and balanced.
- [ ] Daily simulation calculates each transaction exactly once.
- [ ] Asset condition, repairs and downtime work.
- [ ] Daily report explains revenue, costs and net cash change.
- [ ] Net worth includes cash, asset value and liabilities.
- [ ] Save/load survives repeated testing.
- [ ] Tutorial explains at least three financial concepts.
- [ ] UI, audio, credits and licenses are complete.
- [ ] Final build tested outside the Editor.

Submission checklist

- [ ] Official jam rules and deadlines rechecked.
- [ ] Correct project submitted to the correct jam.
- [ ] Game page includes description, controls and screenshots.
- [ ] Gameplay video is verified and accessible.
- [ ] GDD and budget are complete and consistent.
- [ ] AI use disclosed as required.
- [ ] Backup copy of all submitted files saved.

Glossary

Term	Plain-language meaning
Asset	Something the business owns, such as a vehicle or building.
Cash flow	Money coming in and going out over a period.
Revenue	Income before deducting costs.
Operating expense	Recurring cost required to run an asset or business.
Net profit / net operating income	Income left after the costs included in the chosen definition.
ROI	Return on investment: a way to compare gain against money invested; specify the period and costs.
Payback period	Estimated time for net cash earnings to recover an initial cost.
Liquidity	How readily the player can access cash to pay obligations.
Diversification	Spreading investment across different asset types or risks.
Liability	A debt or other financial obligation.
Net worth	Assets minus liabilities, including cash in the game's defined model.
MVP	Minimum viable product: the smallest complete version that tests the core idea.
GDD	Game Design Document: the reference for game goals, mechanics, content and production.

Final reminder

Make the smallest version that clearly demonstrates the full loop: invest → operate → earn or lose → maintain → learn → reinvest. A polished and understandable short game is a stronger foundation than a huge unfinished simulation.

Planning document created October 2026. Working title, economy figures and development estimates are illustrative. Confirm current Zoud GameLab requirements directly with the organizers before submitting.